Assessment-Quality Platform

Blueprint Compliance, Novelty Auditing and Reviewer-Adapted Question Bank Platform

Project Report — Prototype

Document-to-question vertical slice, built and verified 13–14 September 2026

Scope of this report	Problem, full-system design, what the prototype implements, how it was verified, results on real course material, deviations and next steps
Prototype window	14–18 September 2026 (plan §7, §10)
Source plan	`plan.md` (repository root)
Status	End-to-end journey working on one course; automated end-to-end test passing; live generation run completed on real material

Contents

1. Executive summary
2. Problem statement
3. Users and roles
4. The full system (design) and the prototype (built)
5. System architecture
6. Ingestion and retrieval
7. Blueprints
8. Generation orchestrator and LLM backends
9. Blueprint compliance engine
10. Grounding and citations
11. Review workflow and question bank
12. Data model and API
13. User interface
14. Verification and results
15. Design decisions and deviations from the plan
16. Limitations and out-of-scope items
17. Next steps
18. Appendix A — Running the prototype
19. Appendix B — Demo script

1. Executive summary

Faculty write exam papers by hand and verify by memory that each question maps to a course outcome, sits at the right Bloom level, carries the right marks, can be answered from what was taught, and does not repeat a past paper. Generic AI tools produce fluent questions but guarantee none of these properties.

This project builds a platform that **generates questions under explicit constraints and audits every draft before a human sees it**. The full design has three engines: a *blueprint compliance engine*, a *multi-signal novelty engine*, and a *reviewer-adapted ranker*, plus a rigorous evaluation protocol with baselines, ablations, blinded reviewers and bootstrap confidence intervals.

The prototype delivers the first engine end to end on one course: a coordinator uploads course material and freezes a versioned blueprint; a reviewer triggers generation; the system retrieves passages from the uploaded material, drafts a candidate through a swappable LLM backend, runs six deterministic blueprint checks, repairs failing drafts within a budget, and shows each candidate with a pass/fail checklist, draft history and the cited passages with page numbers; the reviewer accepts, edits or rejects; accepted questions are published to a versioned bank with full provenance.

> **What was verified.** An automated end-to-end test runs the whole journey against a real PostgreSQL/pgvector database. A live run on a real 26-page slide deck produced six candidates in 6

> min 40 s on a local open-weight model; four passed every check, and the engine caught Bloom-level, marks and grounding violations on the other two. The generation model never grades its own output.

2. Problem statement

Three failures recur when papers are set manually:

1. **Blueprint drift.** Questions do not fully cover the stated outcomes or land at the wrong Bloom level, and nobody notices until after the exam.
2. **Novelty failure.** A question that has been paraphrased, renumbered or had its conditions reordered reads as new but is substantially a duplicate of a past-year question.
3. **Lost reviewer knowledge.** Every semester starts from scratch; there is no record of which questions were accepted, rejected or edited, or why.

Tools such as ChatGPT, NotebookLM or a basic retrieval-augmented pipeline do not check a blueprint, do not compare against institutional past papers, and do not learn from reviewer behaviour. The project's contribution is the auditing and learning layers around generation, not the generation itself.

3. Users and roles

Actor	Responsibility	Prototype steps
Course Coordinator	Uploads course material; defines, validates and freezes blueprints	1 Upload, 2 Blueprint
Faculty Reviewer (×2)	Triggers generation; accepts, edits or rejects candidates; publishes to the bank	3 Generate, 4 Review, 5 Bank
Research Evaluator	Runs baselines and ablations, reads the evaluation dashboard	Not in prototype

The prototype seeds one course, one coordinator and two reviewers, matching plan §2.

4. The full system (design) and the prototype (built)

Capability	Full system (plan §3)	Prototype status
Ingestion of course material	Parse, chunk, embed, index PDF/Word	**Built and verified on real material**
Versioned blueprints	Draft, validate coverage, freeze as immutable version	**Built**
Grounded generation	Retrieve passages, draft through adapter, cite sources	**Built; Ollama and Anthropic adapters**
Blueprint compliance engine	Rule-based checks + Bloom classifier; repair loop	**Built; six checks, repair budget 2**
Novelty engine	Cosine + BM25 + structural + numerical signals, logistic-regression combiner over labelled pairs	Designed, not built (plan §7.2)
Reviewer-adapted ranker	LightGBM on review records vs two floor baselines	Stub ordering (blueprint position, then compliance)
Review records	Decision, reviewer, edited text, edit distance, reason, timestamp	**Built**
Question bank	Versioned snapshots with provenance	**Built**
Evaluation harness	Baselines, ablations, blinding, seeds, bootstrap CIs	Protocol pre-registered in plan §3.3; schema supports it
Authentication	Institutional SSO via OIDC	OIDC endpoints wired; dev login for seeded users since no IdP is available

Everything scoped out builds on the same pipeline: the novelty engine hooks into the orchestrator after the compliance step, the ranker reads the review record store, and the hosted adapter already implements the same interface as the self-hosted one.

5. System architecture

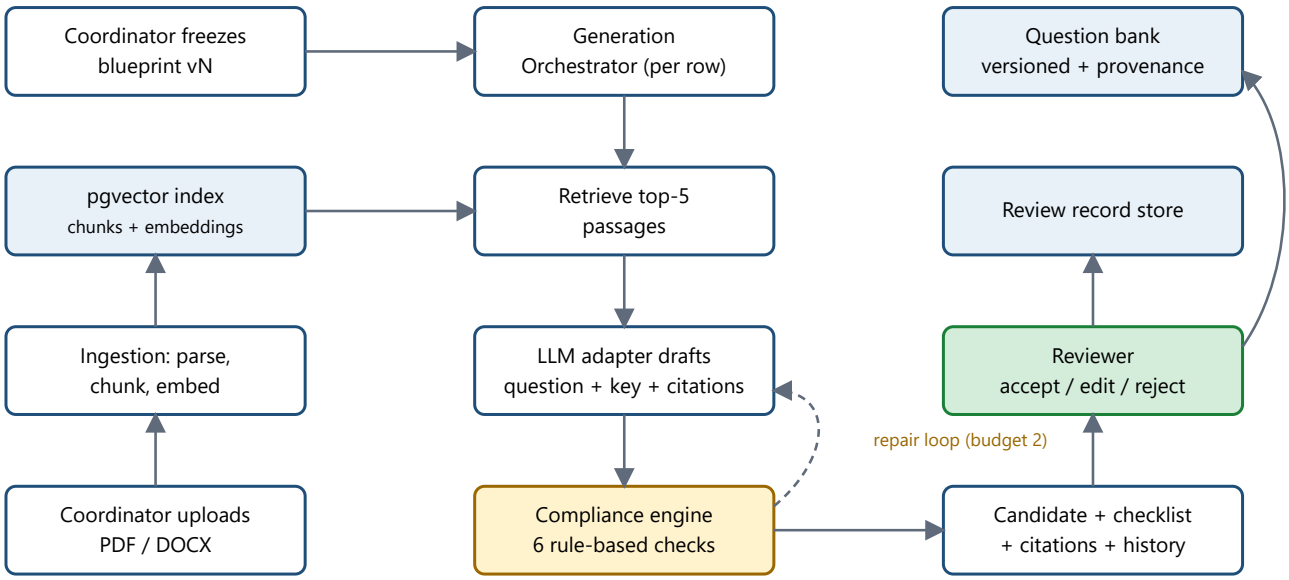


Figure 1 — Prototype pipeline. The novelty engine and ranker (full system) attach after the compliance engine.

Services (plan §6.2) as implemented

Service	Implementation
API gateway	FastAPI application with bearer-token sessions and role checks (`app/auth.py`)
Ingestion	`services/ingestion.py`, `services/embeddings.py`
Blueprint service	`routers/blueprints.py` — draft, constraints, validate, freeze
Generation orchestrator	`services/orchestrator.py` — retrieve → draft → audit → repair, background task per run
Blueprint engine	`services/compliance.py`
Review service	`routers/reviews.py`, `services/review.py` (Levenshtein)
Question bank service	`routers/bank.py`
LLM backend	`llm/base.py` interface; `ollama_adapter.py`, `anthropic_adapter.py`

Data stores: PostgreSQL for all relational tables; the same PostgreSQL with the pgvector extension for chunk embeddings; local disk (`storage/`) for uploaded source files. One database instead of a separate vector store reduces deployment risk (plan §4).

6. Ingestion and retrieval

Parsing

PDFs are read page by page with PyMuPDF; Word files paragraph by paragraph with python-docx. Every extracted sentence carries its locator (`p12` for a PDF page, `para7` for a Word paragraph), which is what later appears as the page reference on a citation.

Real course material is often a slide deck, where bullets rarely end in a full stop. The parser therefore treats bullet glyphs and line breaks as sentence boundaries, re-joins visually wrapped lines (a line starting in lower case continues the previous one), strips leading dash markers, and removes any line that repeats on at least half the pages (headers and footers). On the test deck this took the index from 15 coarse chunks spanning several pages each to 42 chunks with page-accurate references.

Chunking and embedding

Sentences are grouped into overlapping *sentence windows* of 5 with a stride of 3. Each window's page reference spans the first and last sentence (`p3-p4`). Windows are embedded with `sentence-transformers/all-MiniLM-L6-v2` (384 dimensions, L2-normalised) and stored in `chunks.embedding VECTOR(384)`.

Retrieval

For each blueprint row the query is "*concept*. *course outcome*". pgvector returns the five nearest chunks by cosine distance, restricted to the course's materials. The material listing reports the number of chunks indexed and the number of distinct pages covered, so ingestion is visibly real (plan §7.1 step 3), and a passage-preview endpoint shows the first indexed passages with their pages.

7. Blueprints

A blueprint is the exam specification. Each row (constraint) states: course outcome, concept, Bloom level (`remember … create`), marks, question type (`mcq`, `short_answer`, `long_answer`, `numerical`) and target difficulty (`easy`, `medium`, `hard`).

Lifecycle: **draft** (rows editable) → **validate** (marks per outcome are totalled; a row whose difficulty is inconsistent with its Bloom level is flagged — easy pairs with remember/understand, medium with apply/analyze, hard with evaluate/create) → **freeze** (assigns the next version number for the course and locks the rows). Generation only accepts frozen blueprints, and every candidate and bank entry records the blueprint version it came from. This is what makes the evaluation protocol's pre-registration (plan §3.3.1) enforceable rather than a convention.

8. Generation orchestrator and LLM backends

A run is created for a frozen blueprint and executed as a background task. For each row:

1. Retrieve the top-5 passages.

2. Build the prompt from fixed templates: system prompt (write one question strictly from the passages, cite by passage number with an exact quote, invent nothing) and a user prompt listing the constraint fields, type-specific rules, the numbered passages and — on repairs — the exact violations from the previous draft.
3. Call the adapter with a JSON schema: `stem`, `answer_key`, `options[]`, `citations[{passage, quote}]`. Output is schema-constrained, so parsing is strict.
4. Attach citations, run the compliance engine, store the report.
5. If violations remain and the repair budget (2) is not exhausted, redraft with the violations named; otherwise stop.

Each run logs `backend_id`, the SHA-256 of the prompt templates (`prompt_hash`), `temperature` and `seed` — the variance-control fields of plan §3.3.3 — from day one.

Adapters

	Ollama (self-hosted, default)	**Anthropic (hosted)**
Model used	`qwen3:8b`, 4-bit, CPU	`claude-opus-5` (configurable)
Cost	None after a one-time 5 GB download	Per-token API pricing
Structured output	JSON schema via `format`	JSON schema via `output_config.format`
Temperature / seed	Honoured and logged (0.7 / random per run)	Not accepted by current Claude models; logged as NULL rather than a value the backend ignored
Speed on this machine	≈ 40 s per draft	Seconds, network-bound

The orchestrator depends only on the `LLMAdapter` interface, so swapping backends is a configuration change, which is the point of plan §4's adapter decision.

9. Blueprint compliance engine

The engine is deliberately rule-based: deterministic, auditable, and independent of the model that wrote the draft. It runs the following checks against the constraint row.

Field	Rule	Reported as
`bloom_level`	A Bloom classifier scans the stem for cue verbs from a committed list per level (e.g. *define/list* → remember, *explain/describe* → understand, *calculate/apply* → apply, *compare/differentiate* → analyze, *evaluate/justify* → evaluate, *design/propose* → create). The earliest cue in the stem decides; ties go to the higher level. Result must equal the constraint.	expected level, observed level (or *unclassified*)
`question_type`	MCQ: exactly four options and the answer key is one of them. Non-MCQ: no options. Numerical: digits in both stem and answer key.	expected structure, observed structure
`marks`	Answer-key word count must fall in the band for the marks: 1–2 marks → 1–80 words; 3–5 → 20–250; 6–10 → 60–600; above → 200+. Skipped for MCQ.	band, observed word count
`concept`	At least one content word of the concept appears in stem or answer (prefix match).	concept, "terms absent"
`course_outcome`	Same test against the outcome text.	outcome, "terms absent"
`grounding`	At least one citation whose quoted text is found *verbatim* in the cited chunk and whose support score (cosine similarity between the drafted question and the chunk) is at least 0.25.	floor, best verbatim support or "no citation with a verbatim quote"

Each report records `is_compliant` and `satisfaction_rate` = checks passed ÷ checks run, with one row per violation naming field, expected and observed. **The first-draft report is never overwritten by a repair**, so the pre-repair constraint satisfaction rate — the metric in plan §3.3.6 — remains measurable after the fact.

`target_difficulty` has no deterministic rule that can be applied to a candidate's text without inventing one, so it is not checked on the candidate; it is validated for consistency with the Bloom level when the blueprint is validated (Section 7).

10. Grounding and citations

Every candidate stores one `grounding_citation` per cited passage: the chunk id (hence the material and page), the character span where the quoted text was found in the chunk (or `not_found`), and a support score. In the interface the quoted span is highlighted inside the passage so a reviewer sees exactly what evidence the model claimed.

A quote that is paraphrased or invented is stored with `not_found` and does not satisfy the grounding check. In the live run this caught a draft whose citation did not match the passage text and sent it back for

repair (Section 14). This is the mechanism that makes "answerable from what was taught" a checked property rather than a prompt instruction.

11. Review workflow and question bank

Reviewers see candidates only after the run completes and every draft has been audited. For each candidate they can:

- **Accept** as-is.
- **Edit** the wording; the edited stem is stored with the normalised Levenshtein distance (distance ÷ length of the longer string, in [0, 1]).
- **Reject** with a mandatory reason.

Every decision writes a `review_record` (question, reviewer, decision, edited text, edit distance, reason, timestamp, blinded flag). A decision can be changed until the question is published; the summary counts the latest decision per candidate and the mean edit distance.

Publishing takes accepted and edited candidates and writes a `question_bank` entry with a version number and a JSON snapshot: the published stem (edited if edited) and original, answer key, options, the full constraint row, blueprint version and freeze time, run id, backend, prompt hash, temperature and seed, every citation with material and page, the compliance report and repair count, and the reviewer decision. Re-publishing the same question creates a new version; nothing is changed in place.

12. Data model and API

Tables (plan §8)

Table	Purpose	Notes
users	Coordinator and reviewers	Referenced by review_records.reviewer_id
courses, materials, chunks	Course, uploaded files, indexed passages with VECTOR(384)	chunks.page_ref is the citation locator
blueprints, blueprint_constraints	Versioned specification and its rows	version assigned at freeze; rows carry a position
generation_runs	One run per generate call	backend_id, prompt_hash, temperature, seed, status, error
candidate_questions	One per constraint per run	state machine: drafted → compliant/violating → accepted/edited/rejected → published; arm_label reserved for evaluation arms
grounding_citations	Cited chunk, char span, support score	Replaced on each repair
compliance_reports, constraint_violations	One report per draft; one row per violated field	First-draft report preserved
review_records	Reviewer decisions	blinded flag for evaluation sessions
question_bank	Versioned JSONB snapshots	Full provenance

Endpoints (plan §9, all implemented)

```
POST /auth/login            GET  /auth/callback          GET /auth/me   GET /auth/users
GET  /courses
POST /courses/{id}/materials             GET /courses/{id}/materials     GET /materials/{id}/chunks
POST /courses/{id}/blueprints            GET /courses/{id}/blueprints    GET /blueprints/{id}
PUT  /blueprints/{id}/constraints        POST /blueprints/{id}/validate  POST /blueprints/{id}/freeze
POST /blueprints/{id}/generate           GET /blueprints/{id}/runs
GET  /runs/{id}             GET /runs/{id}/candidates    GET /candidates/{id}/evidence
POST /candidates/{id}/review             GET /runs/{id}/reviews
POST /candidates/publish    GET /bank/entries            GET /bank/entries/{id}
```

Role enforcement: uploads, blueprint edits and freezing require *coordinator*; generation, review and publishing require *reviewer*. Read endpoints are open to any signed-in user.

13. User interface

The React front end is a five-step guided flow. A step bar shows which steps are complete, which role performs each, and where the course currently is; every screen opens with a two-sentence "what happens here" and ends with a "Next" action.

Step	Screen	What the user sees
1 Upload material	Coordinator	Upload control; table of files with passages indexed and pages covered; "show sample passages" reveals indexed text with its page.
2 Define blueprint	Coordinator	Blueprint versions; editable row table with a field guide; Save & validate; Freeze enabled only when validation passes; frozen versions visibly locked.
3 Generate	Reviewer	The four pipeline stages explained; Start run; runs table with progress n/total, model, temperature, seed and prompt fingerprint.
4 Review	Reviewer	Per candidate: constraint chips; "passed all 6 checks" or "failed k of 6"; six pass/fail tiles with expected vs found; draft history timeline; source passages with page, support meter and highlighted quote; Accept / Edit / Reject; run summary; Publish.
5 Question bank	Everyone	Published questions with a readable provenance panel: blueprint row and version, run, review, checks at publication, grounded-in pages, answer key.

14. Verification and results

14.1 Automated end-to-end test

`backend/tests/test_e2e.py` runs the complete journey against a real PostgreSQL/pgvector database (`aqp_test`) with a stub LLM adapter whose first draft deliberately violates the Bloom level and cites a non-verbatim quote, and whose repair draft complies. It asserts: a real PDF is parsed and indexed with page references; role checks; blueprint validation catches an inconsistent row and freezing is refused until fixed; freezing assigns version 1 and locks the rows; generation completes with backend id, 64-character prompt hash, temperature 0.7 and an integer seed; exactly two drafts were requested; the first report names `bloom_level` and `grounding` violations and the second is compliant; the citation resolves to page 1 with a found span; an edit stores a normalised edit distance and the summary reflects it; publishing snapshots the edited stem, original stem, blueprint version, run id, citation page and decision; a published question cannot be published again. **Result: passing** (≈20 s after the embedding model is cached).

14.2 Live run on real course material

Material	*Process Modeling and Data Flow Diagrams* — 26-page slide deck (PDF, 1.1 MB)
Ingestion	128 sentences → 42 chunks; 26 of 26 pages covered; repeated footer removed
Retrieval spot-check	"rules for balancing DFDs when decomposing" → top hits p3–4 (0.75), p25–26 (0.73), p22–23 (0.70); "how is a data store drawn" → p7 data-store passage among top 3
Blueprint v1	6 rows, 2 course outcomes, 28 marks; Bloom remember/remember/understand/apply/analyze/evaluate; types short/MCQ/short/short/long/short
Backend	`ollama:qwen3:8b`, temperature 0.7, seed 1519473533
Drafts	10 (6 first drafts + 4 repairs)
Wall-clock	6 min 40 s on CPU, no GPU
Outcome	4 of 6 compliant on the first draft; 4 of 6 compliant after the repair budget

Per-candidate outcome

Q1 — CO1 · data store notation · remember · 2 marks · short answer ✓ compliant, first draft

"Identify the notation used to represent a data store in data flow diagrams." Answer: "A data store is drawn as two horizontal parallel lines." Cited p6–7, support 0.83, quote found verbatim.

Q2 — CO1 · source and sink · remember · 1 mark · MCQ ✓ compliant, first draft

"Identify the correct description of a source/sink in data flow diagrams." Four options, answer key verbatim among them. Cited p8–9 (0.85) and p9–10 (0.75).

Q3 — CO1 · context vs level-1 diagram · understand · 5 marks · short answer ✗ violating after 2 repairs

"Differentiate between a context diagram and a level-1 diagram based on the number of data flows." Draft 1 failed `bloom_level` (expected understand, observed analyze — "differentiate" is an analyze cue). Draft 2 failed bloom_level and `grounding` (quote not verbatim). Draft 3 failed bloom_level and `marks` (19-word answer, band 20–250). Shown to the reviewer with both failures named. Cited p17–18, 0.84.

Q4 — CO2 · balancing DFDs · apply · 5 marks · short answer ✗ violating after 2 repairs

"Balance the following DFD by ensuring the inputs and outputs to the process are conserved at the next level of decomposition." All three drafts failed `bloom_level` as *unclassified*; draft 3 also failed `grounding`. Cited p14–15, 0.83, quote not found.

Q5 — CO2 · DFD rules · analyze · 10 marks · long answer ✓ compliant, first draft

"Analyze the rules for decomposing data flow diagrams into lower-level diagrams and explain how these rules ensure consistency and accuracy in process modeling." 101-word answer key; three citations (p25–26, p11–12, p3–4), support 0.73–0.77.

Q6 — CO2 · functional decomposition · evaluate · 5 marks · short answer ✓ compliant, first draft

Scenario question: a process receiving customer orders decomposed into validation, processing and fulfilment — "Evaluate whether the following DFD decomposition is valid based on the functional decomposition rules provided …". Two citations (p11–12 at 0.69, p18 at 0.43).

Reading the result

- **The audit catches real violations.** Q3's "differentiate" genuinely is an analyze-level verb where understand was specified; the 19-word answer genuinely is too thin for 5 marks; and the non-verbatim quotes on Q3 draft 2 and Q4 draft 3 are exactly the paraphrased-evidence failure the grounding rule exists to catch.
- **One false alarm.** Q4's "Balance the following DFD…" is a legitimate apply-level instruction for this topic, but "balance" is not in the committed cue list, so the classifier reports *unclassified*. Plan §13.2 anticipates this ("usually a Bloom-level misclassification") and asks for such cases to be reported, not hidden. The cue list was **not** extended after seeing this output, because plan §3.3.1 and §5 require the rule set to be committed before evaluation rather than tuned to observed results. Whether to extend it is a decision to take now, before evaluation begins.
- **Repairs help but do not always converge.** The model fixed the grounding failure on Q3 between drafts 2 and 3 but introduced a marks failure, and never changed its verb. A larger repair budget or a stronger model would likely resolve Q3; Q4 cannot pass without a rule change.
- **These are six questions from one run.** They demonstrate that the pipeline is real and that the audit is doing work; they are not a statistical claim. The pre-registered evaluation (baselines, ablations, blinded reviewers, three seeds per constraint, bootstrap intervals) is defined in plan §3.3 and is scheduled after the prototype.

> **Review and publication on the live run** are intentionally left to the faculty reviewers: accept/edit/reject decisions are real review records and future ranker training data, and should not be fabricated by the development team. The review and publish path is verified by the automated test.

15. Design decisions and deviations from the plan

Decision	Reason
Self-hosted Ollama adapter wired in the prototype (plan scheduled it for week 6); hosted Anthropic adapter also implemented	No hosted API credential was available and the mentor's requirement was grounding and checks around whatever model generates. Ollama is free, needs no key, honours temperature and seed, and implements the same interface. Both adapters exist; the choice is one configuration line.
Temperature/seed logged as NULL for the Anthropic adapter	Current Claude models reject sampling parameters and the API has no seed. Logging a value the backend never honoured would misrepresent reproducibility; NULL is honest.
`target_difficulty` not checked on candidates	No deterministic rule exists over the candidate text; inventing one would violate the "rule-based, auditable" principle. It is validated for Bloom consistency on the blueprint instead.
Grounding requires a verbatim quote	Tightened after the mentor's emphasis on grounding. A citation whose quote is not in the passage is now a named violation and triggers a repair.
Slide-aware parsing	Real course material was a slide deck; the plain sentence splitter produced page references spanning several slides, which would have made citations imprecise.
Dev login alongside OIDC	No institutional identity provider is reachable. OIDC endpoints are implemented; with none configured, the login screen lists the seeded users. Switching to SSO is configuration only.
Blueprint `version` nullable until freeze	A draft is not a version. The number is assigned at freeze from the course's latest version + 1.
Ranker is a stub ordering	Plan §7.2: no review records exist yet to fit anything meaningful. Candidates are ordered by blueprint position, then compliance.

16. Limitations and out-of-scope items

- **Novelty engine** — not built; no PYQ ingestion or duplicate scoring in this window.
- **Learned ranker, baselines, ablations, blinding, variance control** — protocol pre-registered and supported by the schema (`arm_label`, `blinded`, temperature, seed), not executed.
- **One course, one blueprint per demo** — the platform is multi-course by schema, but the prototype seeds and demonstrates one.
- **Bloom classifier is keyword-based** — deterministic and committed, with a known false-alarm mode (Section 14). A labelled-example classifier with the 60/20/20 protocol is an option in plan §5.
- **Marks check is a length band** — a proxy for weighting, not a judgement of depth.
- **Local model quality** — an 8B model on CPU produces more repair attempts and will produce more reviewer edits than a frontier model; the evaluation should pin one model and stick with it.

- **Single-process background runs** — generation runs as a FastAPI background task, adequate for a demo, not for concurrent institutional load.

17. Next steps (plan §11)

1. **Decide the Bloom cue list** now, before any evaluation, and commit it.
2. **Reviewer sessions** on the live run to start accumulating review records.
3. **Duplicate-pair labelling** (weeks 2–5): construct hard negatives from real PYQs; 300 pairs minimum; commit the 60/20/20 split before training.
4. **Novelty engine** (weeks 3–5): PYQ ingestion with metadata, four-signal scorer, logistic-regression combiner tuned on validation only, wired after the compliance step, novelty panel in the review screen.
5. **Ranker** (weeks 5–7): LightGBM on pre-review features only, compared against logistic-regression and compliance-only floors; ship the simpler model if it is not beaten.
6. **Backend parity** (week 6): verify identical behaviour across adapters on the same seed.
7. **Blinded evaluation and ablations** (weeks 7–9), **dashboard** (8–10), **failure analysis** (10), **report** (10–17).

18. Appendix A — Running the prototype

```
Prerequisites: PostgreSQL 18 with pgvector 0.8.6 (built from source on this machine), database aqp;
               Python 3.14; Node 24; Ollama with qwen3:8b pulled (or an ANTHROPIC_API_KEY).

cp .env.example backend/.env                      # LLM_BACKEND=ollama, OLLAMA_MODEL=qwen3:8b
cd backend && python -m venv .venv && .venv/Scripts/activate && pip install -r requirements.txt
cd ../frontend && npm install

cd backend  && uvicorn app.main:app --reload --port 8000
cd frontend && npm run dev                        # http://localhost:5173

cd backend && .venv/Scripts/python -m pytest -q   # end-to-end test on database aqp_test
```

Seeded users: `coordinator@example.edu`, `reviewer1@example.edu`, `reviewer2@example.edu`. Repository layout, endpoints and configuration are documented in `README.md`; the full plan in `plan.md`.

19. Appendix B — Demo script (plan §10, Day 5)

Step	Who	Action	What to point at
1	Coordinator	Log in; Upload material; upload the DFD slide deck	"42 passages indexed, 26 pages covered"; show sample passages with page numbers
2	Coordinator	Define blueprint; show version 1 (frozen)	Rows across two outcomes and five Bloom levels; locked badge; explain that generation is only allowed against a frozen version
3	Reviewer 1	Log in; Generate; open the completed run (or start a fresh one if time allows — ≈7 min)	Four pipeline stages; backend, temperature, seed, prompt fingerprint on the run row
4	Reviewer 1	Review; open Q1 then Q3	Q1: all six tiles green, highlighted quote on p6–7. Q3: red tiles naming expected vs found; draft history showing what each repair fixed and broke; "quoted text not found" on draft 2
5	Reviewer 1	Open Q4	The false alarm — explain why it is reported rather than patched (pre-registration)
6	Reviewer 1	Accept Q1, edit Q5's wording, reject Q4 with a reason; Publish	Summary counts and mean edit distance update; publish button names how many will go
7	Anyone	Question bank; show provenance on the edited question	Original vs edited stem, edit distance, blueprint v1, run id, model, seed, grounded-in pages, checks at publication